

### 1. Proje Adı: Kişisel Finans ve Bütçe Takip Uygulaması

**2. Proje Amacı:** Bu uygulama, kullanıcıların gelir ve giderlerini dijital bir ortamda kayıt altına almalarını, harcamalarını kategorize etmelerini ve bütçelerini yönetmelerini sağlamak amacıyla geliştirilmiştir. Kullanıcı dostu bir arayüz ile finansal verilerin kalıcı olarak saklanması hedeflenmiştir.

### 3. Fonksiyonel Gereksinimler:

- Veri Girişi:** Kullanıcı miktar, kategori ve işlem tipi (gelir/gider) bilgilerini sisteme girebilmelidir.
- Veri Listeleme:** Girilen tüm finansal işlemler ana ekrandaki tabloda tarih ve tip bilgisiyle listelenmelidir.
- Veri Saklama:** Uygulama kapatıldığında veriler kaybolmamalı, SQLite veritabanında güvenli bir şekilde depolanmalıdır.
- Kullanıcı Bildirimi:** İşlem başarıyla eklendiğinde veya hatalı veri girildiğinde kullanıcıya mesaj gösterilmelidir.

### 4. Fonksiyonel Olmayan Gereksinimler:

- OOP Uyumluluğu:** Sistem, nesne yönelimli programlama prensiplerine (Kalıtım, Kapsülleme) uygun olmalıdır.
- Hata Yönetimi:** Geçersiz veri girişlerinde program çökmek yerine hata yakalama (try-except) mekanizmalarını kullanmalıdır.
- Modülerlik:** Kod yapısı; arayüz, iş mantığı ve veri erişim katmanı olarak birbirinden ayrı klasörlenmelidir.

---

## UML Diyagramları

Bu bölüme aşağıdaki açıklamaları yazıp, yanlarına basit kutucuk ve oklarla çizimler ekleyebilirsiniz:

### A) Class Diagram (Sınıf Diyagramı)

Kodumuzun iskeleti şu şekildedir:

- Base Class (Temel Sınıf):** Islem (Miktar, Kategori, Tarih özelliklerini taşır).
- Sub Classes (Alt Sınıflar):** Gelir ve Gider (Islem sınıfından kalıtım alarak özelleşirler).
- Manager Class:** DBManager (Veritabanı CRUD işlemlerini yönetir).

### B) Use Case Diagram (Kullanım Durumu Diyagramı)

Kullanıcının sistemle etkileşimi:

- **Aktör:** Kullanıcı.
- **Eylemler:** İşlem Ekleme, Bütçe Görüntüleme, Veri Kaydetme.